

Trident: Efficient FPGA Acceleration of XMSS Tree in Post-Quantum Signature Scheme SLH-DSA

Tianyou Bao¹, Joshua Ennis², Kirill Morozov², and Jiafeng Xie^{1,*} (*: corresponding author)

¹: Department of Electrical and Computer Engineering, Villanova University, Villanova, 19087 USA

²: Department of Computer Science and Engineering, University of North Texas, Denton, TX 76207 USA

¹: Email: {tbao,jiafeng.xie}@villanova.edu; ²: Emails: JoshuaEnnis@my.unt.edu; Kirill.Morozov@unt.edu

Abstract—The emergence of quantum computing poses significant threats to conventional cryptographic systems, necessitating the efficient hardware acceleration of Post-Quantum Cryptography (PQC), especially on the Field-Programmable Gate Array (FPGA) platforms. SPHINCS⁺, recently standardized by NIST (National Institute of Standards and Technology) as SLH-DSA (Stateless Hash-Based Digital Signature Algorithm), represents the only hash-based digital signature scheme. Its practical deployment, however, is restricted by computationally intense operations, particularly in the eXtended Merkle Signature Scheme (XMSS) tree, where WOTS+ (Winternitz One-Time Signature Plus) public key generation consumes the majority of signature generation cycles. With this background, this paper presents Trident, an innovative FPGA-based hardware accelerator that addresses critical performance and resource challenges in XMSS of SLH-DSA. First, we propose a triangle hash unit architecture that enables parallel execution of up to three hash operations simultaneously, directly addressing the computational bottleneck in XMSS tree construction and WOTS+ chain operations. Second, we develop an optimized memory caching scheme that reduces on-chip memory requirements via intermediate value management. Third, we implement the Trident on FPGAs and comprehensively evaluate it across all parameter sets at multiple security levels, i.e., up to $8.6\times$ improvement in signature generation and up to $5.4\times$ speed-up in verification operations. Extended Hypertree evaluation shows a $34.6\times$ area-delay product (ADP) improvement on UltraScale+ FPGA for SLH-DSA-128s. This Trident represents a significant advancement toward practical SLH-DSA deployment in FPGA environments.

Index Terms—FPGA, hardware accelerator, PQC, SLH-DSA, SPHINCS⁺, XMSS, WOTS+.

I. INTRODUCTION

The advancement of quantum computing poses significant threats to conventional cryptographic systems, particularly RSA and Elliptic Curve Cryptography (ECC) [1]. In response, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardization process [2], and selected five algorithms: KYBER and Hamming Quasi-Cyclic (HQC) for key encapsulation, and Dilithium, FALCON, and SPHINCS⁺ for digital signatures [3]. As the only hash-based scheme [4], NIST published FIPS 205, standardizing SPHINCS⁺ as the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA) [5].

Brief Background. SLH-DSA supports multiple security levels (Categories 1, 3, and 5), and offers variants optimized

for either signature size or generation speed [5]. Its security derives from the underlying hash function, supporting both SHA-2 (FIPS 180-4) and SHAKE256 (SHA-3, FIPS 202) [6], [7]. The eXtended Merkle Signature Scheme (XMSS) forms the critical component within SLH-DSA [8], [9].

Existing works. Existing SLH-DSA implementations span multiple formats, including software, graphics processing unit (GPU), hardware-software co-design, and full hardware. Gonzalez et al. proposed a software implementation optimized for embedded settings, competitive with lattice-based solutions while maintaining portability [10]. GPU-based implementations fully utilize their parallelism feature for batch operations, so Kim et al. achieved 44,391 signature generations per second at security level 1 on RTX3090 [11], while Wu et al. developed a CUDA-based batch parallel optimization achieving $1.4\times$ – $2.5\times$ acceleration in signature generation and $4.6\times$ – $11.3\times$ acceleration in signature verification [12].

RISC-V-based hardware-software co-design approaches integrate specialized hash with processors. Wang et al.'s XMSS result came from software optimizations and hardware accelerators [13]. Lopez-Valdivieso et al. used Keccak-1600 and Haraka hash to achieve 15-fold and 90-fold improvements, respectively [14]. Ye et al.'s customized processor with SIMD (Single Instruction Multiple Data) instructions demonstrated $15\times$ acceleration [15]. Zhang et al.'s domain-specific processor achieved $23.1\times$ acceleration in single-core [16].

FPGA-based full hardware designs include: Berthet et al. developed an efficient coprocessor [17]; Amiet et al. achieved $100\times$ speed-up while revealing fault injection vulnerabilities [18]; Deshpande et al.'s SPHINCSLET achieved $4.7\times$ area reduction with $2.5\times$ to $5\times$ signing time improvement [19]; Huang et al. obtained $1.04\times/2.41\times$ area-time-product optimizations [8]; Yoneda et al.'s pipelined XMSS leaf accelerator achieved $20\times$ faster generation speed [9]; and Saarinen et al.'s design achieved two orders of magnitude acceleration [20].

Challenges. FPGA acceleration for PQC represents the recent trend [21] as it offers flexibility and reconfigurability for actual implementation and deployment, which is also recommended by NIST [2], [22]. Compared to GPU implementations, FPGAs provide superior power efficiency and deterministic timing characteristics crucial for real-time cryptographic operations. Furthermore, FPGAs offer significant advantages over Application-Specific Integrated Circuits (ASICs) includ-

ing lower development costs and the ability to support multiple algorithm variants through reconfiguration, making them ideal platforms for evolving PQC standards. However, FPGA acceleration of SLH-DSA faces several critical challenges that limit practical deployment efficiency. i) Memory utilization represents a primary bottleneck, as XMSS tree construction requires substantial intermediate storage for node values and authentication paths; available on-chip memory resources may not be capable of storing all data at once [17]. ii) Hash function’s computational intensity creates performance limitations, particularly in WOTS+ (Winternitz One-Time Signature Plus) public key generation, which accounts for approximately 75% of signature generation cycles [8]. iii) Area-time trade-offs present significant design constraints. High-performance implementations require extensive resources, while compact designs sacrifice throughput [19], [23]. iv) Current accelerators lack comprehensive support for all NIST-specified parameter sets, limiting deployment versatility [8], [17], [24]. These challenges necessitate innovative solutions that balance performance, resource usage, and operational flexibility.

Contributions. This paper thus presents Trident, a novel FPGA-based hardware accelerator approach that addresses the above-mentioned challenges, especially the XMSS, through innovative design solutions. Primary contributions include:

- A triangle hash unit design that enables parallel execution of three hash operations simultaneously. This feature significantly reduces computational latency in XMSS tree structure and WOTS+ chain operations (boost the throughput while maintaining resource efficiency).
- An optimized memory caching scheme that reduces memory resource requirements without external memory dependencies, addressing the critical memory bottleneck in XMSS tree operations through intelligent data reuse and intermediate value management.
- A comprehensive evaluation that validates performance across all NIST-specified parameter sets and security levels in the XMSS structure, demonstrating superior Area-Delay Product (ADP) compared to existing designs.

The remainder of this paper is structured as follows: Section II presents the preliminaries; Section III details the proposed accelerator; Section IV gives implementation results and comparisons; and Section V concludes the work.

II. PRELIMINARIES

Overview. SLH-DSA (SPHINCS⁺) [4], [5] is a stateless hash-based digital signature scheme. SLH-DSA works by signing messages using a Hypertree approach (Fig. 1 and [5]).

Key Generation. This process creates a public key and a private key. In SLH-DSA, the private keys are WOTS+ and FORS (Forest of Random Subsets) keys, and the public key is the root of the XMSS tree at layer $d - 1$.

Signature Generation. It takes a message M and produces a signature that cannot be forged by adversaries. Signatures are generated by hashing the message using a hash function (either SHAKE256 [7] or SHA-2 [6]), and using part of the message digest to choose a FORS key. The FORS signature

of the remainder of the message digest is computed using the chosen FORS key. The full SLH-DSA signature consists of the FORS and the XMSS signatures.

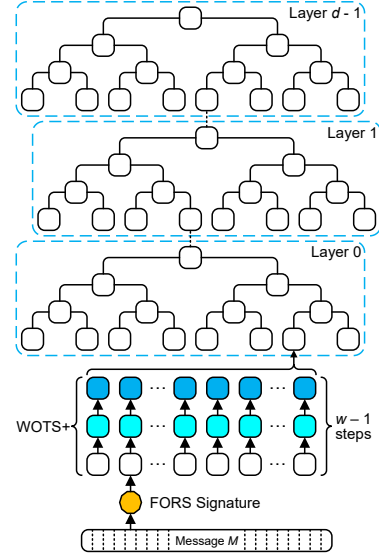


Fig. 1: The structure of the Hypertree in SLH-DSA.

Signature Verification. The SLH-DSA signature consists of a FORS signature, d XMSS authentication paths, and d WOTS+ signatures (part of the XMSS). Authentication of the entire signature consists of verifying the FORS signature, verifying each XMSS authentication path and WOTS+ signature, and calculating the public key. Verification is accepted when the computed public key matches the public key of the sender.

WOTS+. WOTS+ forms XMSS. Its chain function takes an input string X and iteratively hashes it s times (encoding $ADDRS$). WOTS+ is specified by the security n and the Winternitz parameter l_{gw} (always 4 for FIPS 205). A WOTS+ signature is len hash chain digests and is $len \cdot n$ bytes long.

Key Generation. Each hash chain (len chains) is calculated by first generating a secret key sk using a pseudorandom function (PRF). These values are then stored in an array tmp . The hash chains are computed, then compressed using T_{len} , which takes a len input blocks and produces an n -byte output.

Signature Generation. The message M is converted to hex. A three-digit checksum is then computed, and the message (plus checksum) is stored in an array msg of length $len = 2n + 3$. For each element $msg[i]$ ($0 \leq i < len$), a secret key is generated using the PRF and used as the input to the chain function. $msg[i]$ determines the number of iterations. The resulting len hash chains constitute the WOTS+ signature.

Signature Verification. The first part of verification is the same as signing. Next, the hash chains are computed, and the values of $sig[i]$ are hashed $w - msg[i]$ times. The chain is completed, and T_{len} regenerates the public key (verifying).

XMSS. XMSS [5] is a multi-time scheme that uses WOTS+ (Algorithm 1). Since WOTS+ keys only sign one message, the public and private keys need to be regenerated for each signing (inefficient). XMSS uses a Merkle tree so that a single public key can be used to sign $2^{h'}$ messages. The root of the XMSS tree is the public key that is publicly shared. The XMSS

Algorithm 1 $\text{xmss_node}(\text{SK.seed}, i, z, \text{PK.seed}, \text{ADRS})$
Computes the root of a Merkle tree of WOTS+ public keys.

Require: Secret seed SK.seed , node index i , node height z , public seed PK.seed , address ADRS

Ensure: n -byte root node

```

1: if  $z = 0$  then
2:    $\text{ADRS.setTypeAndClear}(\text{WOTS\_HASH})$ 
3:    $\text{ADRS.setKeyPairAddress}(i)$ 
4:    $\text{node} \leftarrow \text{wots\_pkGen}(\text{SK.seed}, \text{PK.seed}, \text{ADRS})$ 
5: else
6:    $\text{lnode} \leftarrow \text{xmss\_node}(\text{SK.seed}, 2i, z-1, \text{PK.seed}, \text{ADRS})$ 
7:    $\text{rnode} \leftarrow \text{xmss\_node}(\text{SK.seed}, 2i+1, z-1, \text{PK.seed}, \text{ADRS})$ 
8:    $\text{ADRS.setTypeAndClear}(\text{TREE})$ 
9:    $\text{ADRS.setTreeHeight}(z)$ 
10:   $\text{ADRS.setTreeIndex}(i)$ 
11:   $\text{node} \leftarrow H(\text{PK.seed}, \text{ADRS}, \text{lnode} \parallel \text{rnode})$ 
12: end if
13: return node

```

signature consists of a WOTS+ signature and h' authentication nodes. The XMSS signature is $(h' + \text{len}) \cdot n$ bytes long.

Signature Generation. As Algorithm 2 shows, XMSS signing consists of choosing a public key at an index idx and computing a WOTS+ signature on M . Then, the h' authentication nodes are computed (Algorithm 1). For each height z , $0 \leq z < h'$, the targeted index i is computed as $\lfloor \frac{i}{2^z} \rfloor \oplus 1$. These nodes $\text{auth}_0, \dots, \text{auth}_{h'-1}$ are the authentication path; XMSS contains both the WOTS+ and the path.

Algorithm 2 $\text{xmss_sign}(M, \text{SK.seed}, \text{idx}, \text{PK.seed}, \text{ADRS})$
Generates an XMSS signature [5].

Require: n -byte message M , secret seed SK.seed , index idx , public seed PK.seed , address ADRS

Ensure: XMSS signature $\text{SIG_XMSS} = (\text{sig} \parallel \text{AUTH})$

```

1: for  $j = 0$  to  $h' - 1$  do
2:    $k \leftarrow \lfloor \frac{\text{idx}}{2^j} \rfloor \oplus 1$ 
3:    $\text{AUTH}[j] \leftarrow \text{xmss\_node}(\text{SK.seed}, k, j, \text{PK.seed}, \text{ADRS})$ 
4: end for
5:  $\text{ADRS.setTypeAndClear}(\text{WOTS\_HASH})$ 
6:  $\text{ADRS.setKeyPairAddress}(\text{idx})$ 
7:  $\text{sig} \leftarrow \text{wots\_sign}(M, \text{SK.seed}, \text{PK.seed}, \text{ADRS})$ 
8:  $\text{SIG\_XMSS} \leftarrow \text{sig} \parallel \text{AUTH}$ 
9: return  $\text{SIG\_XMSS}$ 

```

Signature Verification. XMSS verification is performed by first verifying the WOTS+ signature and regenerating the WOTS+ public key, which is then the leaf at a target index idx and is hashed with auth_0 to generate the first node $n_{\text{idx},0}$ on the authentication path. The hashing is repeated in the same way until the root of the Merkle tree is generated. If the root matches the sender's public key, then the message is verified.

III. PROPOSED ACCELERATOR

A. Top Level Architectural Overview

As Fig. 2 shows, the top-level architecture of the proposed Trident accelerator integrates six primary components: WOTS+ Module, XMSS Module, Triangle Hash Unit, KECCAK, Memory subsystem, and Finite State Machine (FSM). The WOTS+ Module, XMSS Module, and Triangle Hash Unit are interconnected with Data Block RAM (BRAM) through a high-bandwidth memory interface to access data efficiently.

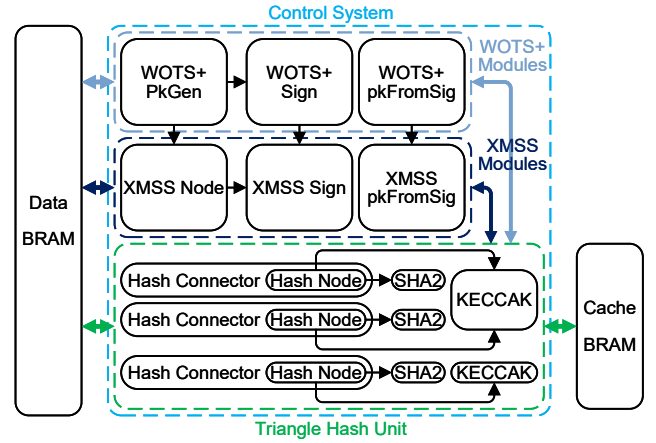


Fig. 2: Top-level overview of the proposed Trident accelerator.

B. Triangle Hash Unit

As shown in Fig. 3, the Triangle Hash Unit includes three independent Hash Connector components, each capable of processing one of three parallel hash operations simultaneously. The 3-way parallelism design represents an optimal balance between resource utilization and performance improvement for XMSS tree operations. While existing works have explored 2-way [19] and 4-way [8] parallel hash architectures, our analysis shows that 2-way approaches incur higher cycle counts for tree construction, whereas 4-way designs consume excessive hardware resources with diminishing performance gains. The evaluation results in Section IV demonstrate that the 3-way approach achieves superior area-delay product across all target FPGA platforms and parameter sets.

Overall Architecture. Each Hash Connector component integrates four critical elements: hash function wrappers for the four hash functions (F , H , PRF , and T_l), Width Align Registers for data alignment, Hash Node for computational processing, and Cache BRAM for intermediate storage. The hash function wrappers format the input data according to the specific hash function definitions, combining elements such as public key seed (PK.seed), private key seed (SK.seed), and message (M) to form the appropriate hash input values.

Width Align Register. The Width Align Register serves as a critical interface component, temporarily storing input data and ensuring proper width alignment between the hash function outputs and the Hash Node's input requirements. For all hash modes, input data is converted into n -byte (or $8n$ -bit) output values. The Width Align Register supports multiple hash core configurations: SHA-256 cores require 512-bit input blocks, SHA-512 cores need 1024-bit input blocks, and KECCAK cores process 1344-bit input blocks. After hash computation, the Hash Node output is processed through the Width Align Register and converted back to the corresponding memory width for Data/Cache Memory interfaces.

Memory Management. The computed hash values are routed to Data BRAM or Cache BRAM based on data dependency. For example, in XMSS tree construction, when parent nodes require hash values from recently computed child nodes, the results are stored in Cache BRAM to enable efficient

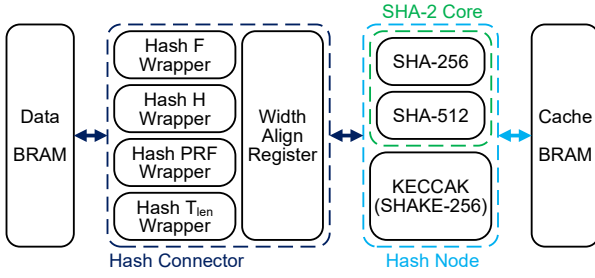


Fig. 3: The structure of the Hash Connector.

data reuse. This caching mechanism significantly reduces Data BRAM access requirements, thereby improving overall system performance and memory bandwidth utilization.

Unlike previous sequential or 2-instance hash processing approaches, the Triangle Hash Unit enables concurrent execution of three hash operations, particularly beneficial for XMSS tree construction, where multiple nodes can be computed simultaneously. This parallel processing capability directly addresses the computational intensity challenge in WOTS+ public key generation and XMSS authentication path construction.

C. Hash Function Wrapper

The hash function wrapper supports various input formatting and parameter configurations. The **F** function processes an n -byte message with public key seed and address to produce an n -byte output for WOTS+ chain operations. The **H** function accepts a $2n$ -byte message to generate an n -byte hash for Merkle tree node computation. The **PRF** function generates n bytes of pseudorandomness for secret key generation. The **T_l** function implements variable-length message compression for WOTS+ public key compression. Each wrapper handles input concatenation per SLH-DSA specification.

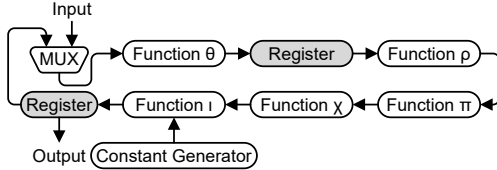


Fig. 4: Overall architecture of the optimized KECCAK.

D. Optimized KECCAK Unit

The optimized KECCAK unit addresses the performance bottleneck in hash-intensive operations through a 2-stage pipelined architecture, which can process two SHAKE hash operations concurrently. This design maximizes hardware resource utilization while significantly improving throughput compared to conventional sequential implementations.

Pipelined Architecture. As shown in Fig. 4, the optimized KECCAK unit is a 2-stage pipelined structure where each stage can independently process different hash operations. The first stage handles the θ transformation, which performs column parity computation and XOR operations across the state array. Input values enter the θ function during the first clock cycle and are subsequently stored in intermediate registers. The second stage sequentially processes the ρ (rotation), π (permutation), χ (non-linear transformation), and ι (round constant addition) functions during the second clock cycle.

Critical Path Optimization. This pipelined approach effectively balances the critical path across both clock cycles, enabling higher throughput while maintaining the same operating frequency. The separation of θ from the remaining permutation functions addresses the inherent timing imbalance in the KECCAK- $f[1600]$ permutation, where θ operations require a major delay due to their global state dependencies. Our post-synthesis timing analysis revealed that the θ transformation computational delay approximately equals the combined delay of the ρ , π , χ , and ι functions, which motivated the specific 2-stage partitioning strategy. Experimental evaluation of alternative pipeline configurations (e.g., $\theta + \rho$, $\theta + \rho + \pi$) demonstrated that this 2-stage approach achieves the optimal operating frequency for both Artix-7 and UltraScale+ target platforms while minimizing register overhead.

TABLE I: Implementation Comparison of KECCAK

Design	LUT	FF	Slice	Freq (MHz)	Latency	ADP
[19]	4,383	2,708	1,434 ¹	250.00	49 (21/24/4)	562.13 ²
Pro.	6,698	8,751	1,978	250.00	51 (1/49/1)	403.51

¹: Equivalent Slice calculated based on the formula $LUT/4 + FF/8$ [25].
²: ADP for two complete hash cycles; the lower the better.

Performance Comparison. Table I presents a comprehensive comparison between the non-pipelined KECCAK design from [19] and proposed optimized KECCAK. Compared to existing implementations requiring two independent KECCAK units to complete dual hash operations in 49 cycles [19], the proposed unit accomplishes the same task using a single pipeline instance in 51 cycles. The optimized design requires 37.9% additional slice utilization. Overall, the optimized KECCAK achieves a 28.2% improvement in ADP in processing two complete hash cycles. The pipelined structure enables continuous data processing, allowing new hash operations to be performed before the completion of previous operations, effectively doubling the sustained throughput rate.

Resource Configuration. The Triangle Hash Unit incorporates two optimized KECCAK cores with identical 2-stage pipelined architecture. Hash Connectors 0 and 1 share KECCAK Core 0 in an interleaved pattern, exploiting the pipeline stages to process two hash streams concurrently. Hash Connector 2 utilizes dedicated KECCAK Core 1 with matching architecture to maintain timing balance across all three connectors. This asymmetric sharing strategy optimizes resource utilization while sustaining maximum throughput.

E. SHA-2 Core

The Trident accelerator integrates SHA-2 hash functions (SHA-256 and SHA-512) based on security level requirements [5], [6]. The Triangle Hash Unit incorporates three SHA-256 cores (one per Hash Connector) for all security levels, with three additional SHA-512 cores for levels 3 and 5. The cores employ iterative architecture, processing 64 rounds (SHA-256) or 80 rounds (SHA-512) sequentially. The design integrates validated open-source implementations [26], [27] accessed through wrapper components providing uniform $8n$ -bit interfaces consistent with KECCAK cores.

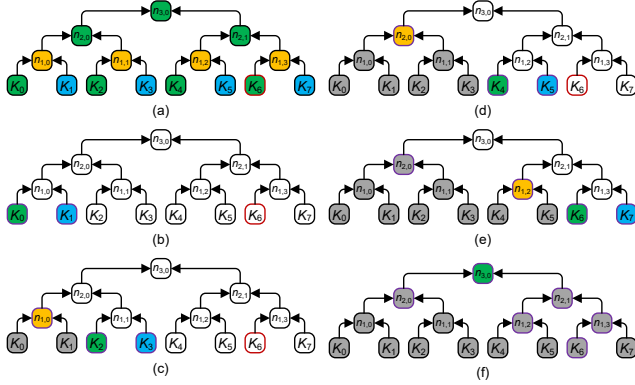


Fig. 5: Data flow within the triangle hash unit.

F. Optimized Memory Caching and Data Flow

The data and cache BRAM are designed to optimize the usage of on-chip memory resources while maximizing the memory bandwidth to boost the throughput. The memory width is configured as n bytes to match the specific requirements of the various parameter sets, with a separate memory space allocated for cache, especially for the hash functions.

XMSS tree construction and WOTS+ public key generation require substantial intermediate value storage. Existing accelerator designs like [8] rely on external memory resources to avoid excessive on-chip memory usage. However, such designs increase system complexity and introduce memory access latency. To address this challenge, we use an optimized memory caching scheme that effectively manages intermediate values and hash function data without external memory dependencies, thereby improving overall system performance.

Triangle Hash Unit Data Flow. Fig. 5 demonstrates the data flow within the triangle hash unit for XMSS tree construction. The color-coded nodes represent different Hash Connector assignments: green nodes are processed by Hash Connector 0, blue nodes by Hash Connector 1, and orange nodes by Hash Connector 2. The red-bordered node K_6 represents the node starting the authentication path, while purple-bordered nodes indicate values stored in Cache BRAM. Fig. 5(a) demonstrates the construction process of an XMSS tree with $h' = 4$, where the value of each parent node is computed by hashing data from its two child nodes.

Parallel Processing Strategy. Fig. 5(b) shows the initial hash operation, where only Hash Connector 0 and Hash Connector 1 process leaf nodes K_0 and K_1 , respectively. The K_0 and K_1 are the nodes from the sibling branch of the targeted node K_6 in the binary tree structure, from the perspective of the root node. The generated hash values are stored in Cache BRAM for subsequent operations. During the second hash cycle, as Fig. 5(c) shows, Hash Connector 2 computes the hash value of $n_{1,0} = K_0 || K_1$ and stores it in Cache BRAM. Meanwhile, Hash Connector 0 and Hash Connector 1 process nodes K_2 and K_3 . This pattern continues as shown in Fig. 5(d) until the left-half subtree processing is accomplished, leaving only node $n_{2,0}$ stored in Cache BRAM. Concurrently, Hash Connector 0 and Hash Connector 1 process the furthest unprocessed nodes K_4 and K_5 relative to the

targeted node K_6 . In the next several cycles, Hash Connector 0 computes K_6 , Hash Connector 1 processes K_7 , and Hash Connector 2 handles nodes $n_{1,2}$, $n_{1,3}$, $n_{2,1}$, and $n_{3,0}$. This coordinated approach effectively reduces Data BRAM access requirements while maintaining high throughput performance. The parallelization efficiency varies with parameter configurations: (i) in small signature mode (with larger h' values), this method achieves higher parallelization rates; (ii) conversely, in fast signature mode (with smaller h' values), the memory access pattern limits the order of data accessing, which reduces parallelization rates for certain operations.

G. FSM

The FSM serves as the central coordination controller for the Trident accelerator, managing the complex interactions between multiple modules while ensuring correct execution order and optimal process latency. The FSM design addresses the intricate dependencies within XMSS signature generation and hash function computations, providing systematic control over the entire signature generation and verification processes.

Flow Control. During XMSS signature generation, the FSM coordinates multiple concurrent operations, i.e., XMSS node computation for authentication path generation and WOTS+ signature generation. The XMSS node operations require coordinated invocation of WOTS+ public key generation and the hash function H for tree node computation. For XMSS signature verification through the pkFromSig operation, the FSM manages WOTS+ public key reconstruction processes. The WOTS+ module operations involve chain function computations and invoking hash functions PRF and T_i , where individual chain operations utilize the hash function F . This hierarchical dependency structure requires precise FSM coordination to maintain operational correctness and efficiency.

TABLE II: SLH-DSA Parameter Sets and Configurations

Parameter Set	n	h	d	h'	a	k	lg_w	m	Sec. Level
SLH-DSA-128s	16	63	7	9	12	14	4	30	1
SLH-DSA-128f	16	66	22	3	6	33	4	34	1
SLH-DSA-192s	24	63	7	9	14	17	4	39	3
SLH-DSA-192f	24	66	22	3	8	33	4	42	3
SLH-DSA-256s	32	64	8	8	14	22	4	47	5
SLH-DSA-256f	32	68	17	4	9	35	4	49	5

When XMSS tree construction is required, the FSM triggers the Triangle Hash Unit through appropriate Hash Connector scheduling, directing computed hash values to Cache BRAM for intermediate storage and reuse. During WOTS+ signature operations, the FSM activates the WOTS+ Module to execute public key generation and signature computations in coordination with the Triangle Hash Unit. The controller maintains strict data dependency tracking to ensure that dependent operations receive correct input data at appropriate timing intervals.

Memory Management. The FSM implements memory access control, managing data flow between Data BRAM, Cache BRAM, Hash modules, and other computational modules. Input data retrieval from the Data BRAM is coordinated with the Hash modules to minimize memory access latency. Intermediate computational results are directed to Cache BRAM when subsequent operations require immediate access, while final results are written back to Data BRAM upon completion.

TABLE III: Implementation Results for XMSS Operation on Proposed Trident Accelerator (SHA-2)

Design	Para. Set ($n/h/w$)	Device	Feature	LUT	FF	Slice/ CLB ¹	BRAM	F_{max} (MHz)	Latency ² Sign / Verify	Delay ³ Sign / Verify	ADP for Sign
[28]	32/10/16	Artix-7 200T	HW	22,300	31,215	12,313	0.0	110	317.00M / 160.00K	2.88K / 1.45	35.48K
[29] S ⁴	32/10/16	Artix-7 75T	HW	13,308	13,625	8,516	15.0	100	468.00M / 250.00K	4.68K / 2.50	39.86K
[29] M ⁴	32/10/16	Artix-7 75T	HW	22,560	19,157	11,521	15.0	100	110.00M / 68.20K	1.10K / 0.68	12.67K
[29] L ⁴	32/10/16	Artix-7 75T	HW	30,643	23,723	14,112	15.0	100	78.30M / 53.00K	783.00 / 0.53	11.05K
[13]	32/10/16	Intel i5	SW	-	-	-	-	3200	5.60B / 1.75K	3.00M / 0.94	-
Pro. ⁵	32/10/16	Artix-7 200T	HW	26,281	45,054	13,248	4.5	100	36.89M / 29.37K	368.86 / 0.29	4.89K
[28]	32/16/16	Artix-7 200T	HW	22,300	31,215	12,313	0.0	110	20.50M / 180.00K	186.36K / 1.64	2.29M
[13]	32/16/16	Intel i5	SW	-	-	-	-	3200	360.00B / 112.50K	3.10M / 0.97	-
Pro. ⁵	32/16/16	Artix-7 200T	HW	26,281	45,054	13,248	4.5	100	2.35B / 30.27K	23.55K / 0.30	311.98K
[28]	32/20/16	Artix-7 200T	HW	22,300	31,215	12,313	0.0	110	327.70M / 191.00K	2.98M / 1.74	36.68M
[13]	32/20/16	Intel i5	SW	-	-	-	-	3200	5.70T / 1.78M	3.20M / 1.00	-
Pro. ⁵	32/20/16	Artix-7 200T	HW	26,281	45,054	13,248	4.5	100	37.49B / 29.88K	374.86K / 0.30	4.97M

¹: Slice/CLB is calculated based on the formula $LUT/4 + FF/8 + DSP \times 102.4 + BRAM \times 232.4$ and $LUT/4 + FF/8 + DSP \times 51.2 + BRAM \times 116.2$ for 7-series and UltraScale+ board, respectively, according to [25] if the work did not report it.

²: Latency refers to the cycles of the whole XMSS operation. ³: Delay=Latency/ F_{max} . Unit: ms.

⁴: S represent BDS k = 8, Minimal (1x1); M represent BDS k = 8, Medium (5x4), L represent BDS k = 8, Large (8x7).

⁵: Since our work is not designed for non-standardized parameter sets, the BRAM usage listed here cannot reflect the real situation under the SLH-DSA parameter sets. Nevertheless, the Latency and Delay calculations are estimated based on the SLH-DSA algorithm.

Note: Related works implement standard XMSS, while our design implements SLH-DSA's stateless XMSS variant. Despite algorithmic differences (e.g., TI function vs. L-tree compression), the hardware resource comparison remains valid as both implementations perform fundamentally similar XMSS tree construction operations with equivalent hash core requirements.

Based on operation requirements and data dependencies, the controller assigns hash computations to available Hash Connectors while maintaining load balancing across parallel processing units. For operations requiring sequential dependency, the FSM ensures proper Hash Connectors are utilized to avoid data collision. During high-parallelism phases, such as XMSS leaf node computation, the controller maximizes parallel operation execution while monitoring memory access patterns to prevent performance degradation.

IV. EVALUATION AND RESULTS

This section presents a comprehensive evaluation, which focuses on three primary metrics: area utilization, timing, and ADP. Area metrics include Look-Up Tables (LUTs), Flip-Flops (FFs), Slices/Configurable Logic Blocks (CLBs), and BRAMs. Timing evaluation encompasses clock cycles (latency), maximum clock frequency (F_{max}), and execution time (ms). The ADP represents the product of execution time and slice utilization, providing a balanced performance-area trade-off metric. Since SLH-DSA supports two hash implementations, we present separate evaluations for both configurations.

Experimental Setup. Our accelerator was coded in VHDL and implemented using AMD-Xilinx Vivado 2022.2, targeting the Artix-7 (xc7a200tffg1156-3) and Ultrascale+ (xczu9eg-ffvb1156-2-e) FPGA platform following [8], [19]. This FPGA platform selection ensures direct comparability with existing literature while representing a cost-effective solution for embedded applications, which is recommended by NIST [22].

The proposed Trident accelerator was ensured with precise control and functionality. Meanwhile, Trident supports all six SLH-DSA parameter sets as listed in Table II, providing comprehensive coverage across all NIST-defined security levels.

Implementation Results. Table III presents performance evaluation results for XMSS operations implemented on the proposed Trident accelerator, comparing against existing state-of-the-art hardware and software implementations across multiple parameter configurations. The evaluation covers three

different tree height parameters ($h = 10, 16, 20$) with the same security parameters ($n = 32, w = 16$) to demonstrate the performance across varying computational complexity levels. Note that our design targets standard SLH-DSA parameter sets, and BRAM utilization may not represent actual deployment scenarios for non-standard configurations. Latency and delay results are derived from the algorithms of SLH-DSA.

The proposed Trident accelerator demonstrates significant area efficiency compared to existing hardware implementations. For the $h = 10$ configuration, our design requires only 8,574 LUTs and 12,981 FFs, consuming 4,202 slices, which represents a 65.9% reduction in slice utilization compared to [28], which requires 12,313 slices. When compared to [29], our design achieves 50.7% fewer slices than the minimal design (S) and 63.5% fewer slices than the large design (L). Notably, BRAM consumption remains relatively consistent across different SLH-DSA security levels (128s/192s/256s) because the memory requirement scales linearly with the XMSS tree height h' rather than exponentially with n . Since all evaluated parameter sets utilize identical h' values within the XMSS layer, the primary memory growth is attributed to increased hash output widths (n bytes). Our architecture uses register storage (FFs) for intermediate value buffering rather than allocating additional BRAMs, which explains the notably higher FF utilization compared to BRAM-intensive designs while maintaining efficient on-chip memory usage.

The timing result reveals substantial improvements in computational efficiency. For signature generation with $h = 10$, our accelerator has a 36.89M cycles latency, corresponding to 335.33 ms execution time at 100 MHz. This represents an $8.6\times$ improvement over [28] and half the delay time against the large designs, which is better than [29]. The verification operations demonstrate even more advantages. The Trident design achieves 29.37K cycles (0.27 ms) delay compared to 1.45 ms for [28], representing a $5.4\times$ speed-up.

The ADP metric provides an easy-to-use and straightforward

TABLE IV: Implementation Results for Hypertree Operation on Proposed Trident Accelerator (SHA-2)

Design	Para. Set	Device	LUT	FF	Slice/ CLB ³	BRAM	F_{max} (MHz)	Latency ¹ Sign / Verify	Delay ² Sign / Verify	ADP for Sign
[19]	SLH-DSA-128s	Artix-7 200T	6,767	6,054	4,308	8.0	110.00	75.11M / 143.94K	682.86 / 1.31	2.94K
[20]	SLH-DSA-128s	Artix-7 100T	5,486	3,675	9,268	32.0	100.00	190.09M / 268.45K	1.90K / 2.68	17.62K
Pro.	SLH-DSA-128s	Artix-7 200T	8,574	12,981	4,347	2.5	110.00	40.99M / 72.14K	372.64 / 0.66	1.62K
[19]	SLH-DSA-128f	Artix-7 200T	6,787	6,083	4,316	8.0	110.00	4.02M / 429.42K	36.59 / 3.90	157.92
[20]	SLH-DSA-128f	Artix-7 100T	5,486	3,675	9,268	32.0	100.00	9.13M / 691.19K	91.27 / 6.91	845.87
Pro.	SLH-DSA-128f	Artix-7 200T	8,599	13,295	4,349	2.5	110.00	2.47M / 177.94K	22.50 / 1.62	98.82
[19]	SLH-DSA-192s	Artix-7 200T	15,615	15,183	8,590	12.0	100.00	139.60M / 358.32K	1.40K / 3.58	12.00K
[20]	SLH-DSA-192s	Artix-7 100T	8,965	6,823	10,531	32.0	100.00	626.86M / 641.05K	6.27K / 6.41	66.01K
Pro.	SLH-DSA-192s	Artix-7 200T	23,333	35,752	11,116	3.5	100.00	98.96M / 151.85K	989.64 / 1.52	11.00K
[19]	SLH-DSA-192f	Artix-7 200T	15,436	15,144	8,541	12.0	100.00	7.20M / 1.09M	72.05 / 10.86	615.33
[20]	SLH-DSA-192f	Artix-7 100T	8,965	6,823	10,531	32.0	100.00	23.73M / 1.29M	237.26 / 12.91	2.50K
Pro.	SLH-DSA-192f	Artix-7 200T	23,222	35,401	11,044	3.5	100.00	6.81M / 463.72K	68.11 / 4.64	752.25
[19]	SLH-DSA-256s	Artix-7 200T	15,618	15,225	8,596	12.0	100.00	127.31M / 539.16K	1.27K / 5.39	10.94K
[20]	SLH-DSA-256s	Artix-7 100T	8,965	6,823	10,531	32.0	100.00	696.20M / 894.08K	6.96K / 8.94	73.32K
Pro.	SLH-DSA-256s	Artix-7 200T	26,272	45,268	13,272	4.5	100.00	74.14M / 229.31K	741.44 / 2.29	9.84K
[19]	SLH-DSA-256f	Artix-7 200T	15,513	15,206	8,568	12.0	100.00	13.87M / 1.12M	138.73 / 11.20	1.19K
[20]	SLH-DSA-256f	Artix-7 100T	8,965	6,823	10,531	32.0	100.00	50.24M / 1.42M	502.41 / 14.19	5.29K
Pro.	SLH-DSA-256f	Artix-7 200T	26,281	45,054	13,248	4.5	100.00	11.72M / 480.42K	117.24 / 4.80	1.55K
[19]	SLH-DSA-128s	UltraScale+ XCZU3EG	7,476	6,120	3,564	8.0	250.00	93.89M / 179.92K	375.57 / 0.72	1.34K
[17]	SLH-DSA-128s	UltraScale+ XCZU3EG	6,072	4,733	2,168	0.5	154.00	- / -	958.16 / 1.12	2.08K
Pro.	SLH-DSA-128s	UltraScale+ XCZU3EG	8,598	13,282	3,194	2.5	250.00	40.99M / 72.14K	163.96 / 0.29	523.70
[19]	SLH-DSA-128f	UltraScale+ XCZU3EG	7,345	6,062	3,524	8.0	250.00	5.03M / 536.77K	20.12 / 2.15	70.90
[17]	SLH-DSA-128f	UltraScale+ XCZU3EG	6,127	4,933	2,206	0.5	156.00	- / -	64.34 / 2.51	141.96
Pro.	SLH-DSA-128f	UltraScale+ XCZU3EG	8,541	12,968	2,151	2.5	250.00	2.47M / 177.94K	9.90 / 0.71	21.29
[19]	SLH-DSA-192s	UltraScale+ XCZU3EG	17,061	15,178	7,512	12.0	230.00	174.50M / 447.90K	758.71 / 1.95	5.73K
Pro.	SLH-DSA-192s	UltraScale+ XCZU3EG	23,282	35,789	5,400	3.5	210.00	98.96M / 151.85K	471.26 / 0.72	2.54K
[19]	SLH-DSA-192f	UltraScale+ XCZU3EG	16,900	15,140	7,557	12.0	230.00	9.01M / 1.36M	39.16 / 5.90	294.13
Pro.	SLH-DSA-192f	UltraScale+ XCZU3EG	23,004	35,461	3,510	3.5	210.00	6.81M / 463.72K	32.43 / 2.21	113.85
[19]	SLH-DSA-256s	UltraScale+ XCZU3EG	16,879	15,217	7,516	12.0	210.00	159.14M / 673.95K	757.80 / 3.21	5.70K
[17]	SLH-DSA-256s	UltraScale+ XCZU3EG	8,612	6,366	3,007	0.5	149.00	- / -	1.73K / 2.46	5.22K
Pro. ⁴	SLH-DSA-256s	UltraScale+ XCZU3EG	-	-	-	-	-	- / -	- / -	-
[19]	SLH-DSA-256f	UltraScale+ XCZU3EG	16,762	15,190	7,484	12.0	210.00	17.34M / 1.40M	82.58 / 6.66	617.99
[17]	SLH-DSA-256f	UltraScale+ XCZU3EG	8,591	6,339	2,998	0.5	152.00	- / -	199.20 / 4.55	597.25
Pro. ⁴	SLH-DSA-256f	UltraScale+ XCZU3EG	-	-	-	-	-	- / -	- / -	-
Pro.	SLH-DSA-128s	UltraScale+ XCZU9EG	8,598	13,282	2,186	2.5	250.00	40.99M / 72.14K	163.96 / 0.29	358.42
Pro.	SLH-DSA-128f	UltraScale+ XCZU9EG	8,553	12,968	2,081	2.5	250.00	2.47M / 177.94K	9.90 / 0.71	20.60
Pro.	SLH-DSA-192s	UltraScale+ XCZU9EG	23,287	35,790	5,667	3.5	200.00	98.96M / 151.85K	494.82 / 0.76	2.80K
Pro.	SLH-DSA-192f	UltraScale+ XCZU9EG	23,098	35,461	5,815	3.5	200.00	6.81M / 463.72K	34.06 / 2.32	198.04
Pro.	SLH-DSA-256s	UltraScale+ XCZU9EG	26,170	45,318	6,570	4.5	200.00	74.14M / 229.31K	370.72 / 1.15	2.44K
Pro.	SLH-DSA-256f	UltraScale+ XCZU9EG	26,170	45,318	6,570	4.5	200.00	11.72M / 480.42K	58.62 / 2.40	385.14

¹: Latency refers to the cycles of the whole Hypertree operation. ²: Delay=Latency/ F_{max} . Unit: ms.

³: Slice/CLB is calculated based on the formula $LUT/4 + FF/8 + DSP \times 102.4 + BRAM \times 232.4$ and $LUT/4 + FF/8 + DSP \times 51.2 + BRAM \times 116.2$ for 7-series and UltraScale+ board, respectively, according to [25] if the work did not report it.

⁴: For SLH-DSA-256s/f design, the UltraScale+ XCZU3EG board does not have enough I/O Pins for implementations, thus, we do not report the result.

ward performance-area trade-off index. Our design achieves 1.41K ADP for $h = 10$ configuration, representing a $25.2\times$ improvement over ADP of work [28], and superior performance compared to [29] (ranging from 11.05K to 39.86K). This substantial ADP improvement showcases the effectiveness of our triangle hashing and optimized memory caching to achieve balanced performance-resource utilization.

The proposed design also outperforms software implementations [13], achieving $153.2\times$ and $63.3\times$ speed-up in signature generation and verification. Power consumption ranges from 0.24W to 3.22W across platforms and parameter sets. Note that direct power comparison with existing works is not feasible as referenced implementations do not report power consumption metrics in their results.

Extended Application. The proposed accelerator primarily implements the XMSS component of SLH-DSA. To demonstrate comprehensive performance comparison against existing state-of-the-art implementations, we extend our XMSS design

to support Hypertree signature and verification operations. This extended application shows the potential of the Trident accelerator architecture beyond its core XMSS functionality.

Since existing works do not provide independent performance reports for the Hypertree operations, we estimate the performance results based on Table III of [8], WOTS+ operations consume approximately 81.25% of total SLH-DSA signature generation cycles and 82.12% of verification cycles. Therefore, we estimate the Hypertree operation latency for existing SLH-DSA/SPHINCS⁺ implementations by applying a scaling factor of 0.8 to their reported total latency values.

Tables IV and V present comprehensive performance evaluation results for complete Hypertree operations across both SHA-2 and KECCAK hash implementations. The evaluation demonstrates the extended capabilities of the Trident.

SHA-2-based Implementation. For SLH-DSA-128s, our design achieves 372.64 ms signature delay ($1.8\times$ speed-up over [19]) with comparable 4,347 slices. Compared to [20],

TABLE V: Implementation Results for Hypertree Operation on Proposed Trident Accelerator (KECCAK)

Design	Para. Set	Device	LUT	FF	Slice/ CLB ³	BRAM	F_{max} (MHz)	Latency ¹ Sign / Verify	Delay ² Sign / Verify	ADP for Sign
[19]	SLH-DSA-128s	Artix-7 200T	10,255	7,410	5,349	8.0	150.00	43.56M / 76.60K	290.40 / 0.51	1.55K
[18]	SLH-DSA-128s	Artix-7 100T	48,231	72,514	23,795	11.5	250.00	- / -	12.40 / 0.07	295.05
[20]	SLH-DSA-128s	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	102.35M / 179.60K	1.02K / 1.80	10.29K
Pro.	SLH-DSA-128s	Artix-7 200T	14,189	23,702	4,984	2.5	220.00	55.08M / 51.29K	250.36 / 0.23	1.25K
[19]	SLH-DSA-128f	Artix-7 200T	10,197	7,357	5,328	8.0	150.00	2.32M / 228.85K	15.44 / 1.53	82.28
[18] ⁴	SLH-DSA-128f	Artix-7 100T	47,991	72,505	23,836	11.5	250.00	- / -	1.01 / 0.16	24.07
[20]	SLH-DSA-128f	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	4.90M / 440.64K	49.04 / 4.41	493.15
[8] ⁵	SLH-DSA-128f	Artix-7 100T	29,410	14,090	10,043	4.0	179.80	349.00K / 27.70K	1.94 / 0.15	19.49
Pro.	SLH-DSA-128f	Artix-7 200T	14,245	24,022	5,045	2.5	220.00	2.90M / 132.66K	13.19 / 0.60	66.54
[19]	SLH-DSA-192s	Artix-7 200T	10,644	7,643	5,476	8.0	150.00	75.79M / 110.25K	505.28 / 0.74	2.77K
[18]	SLH-DSA-192s	Artix-7 100T	48,725	72,514	25,196	17.0	250.00	- / -	21.40 / 0.10	539.20
[20]	SLH-DSA-192s	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	263.10M / 289.83K	2.63K / 2.90	26.46K
Pro.	SLH-DSA-192s	Artix-7 200T	15,505	26,141	5,686	3.5	220.00	70.37M / 61.97K	319.88 / 0.28	1.82K
[19]	SLH-DSA-192f	Artix-7 200T	10,333	7,590	5,391	8.0	150.00	3.73M / 331.49K	24.88 / 2.21	134.16
[18] ⁴	SLH-DSA-192f	Artix-7 100T	48,398	73,476	25,337	17.0	250.00	- / -	1.17 / 0.19	29.64
[20]	SLH-DSA-192f	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	10.60M / 711.43K	105.96 / 7.11	1.07K
[8] ⁵	SLH-DSA-192f	Artix-7 100T	29,410	14,090	10,043	4.0	179.80	571.90K / 40.50K	3.18 / 0.23	31.94
Pro.	SLH-DSA-192f	Artix-7 200T	15,505	26,141	5,686	3.5	220.00	4.23M / 186.24K	19.22 / 0.85	109.26
[19]	SLH-DSA-256s	Artix-7 200T	10,802	7,698	5,522	8.0	150.00	66.84M / 163.28K	445.62 / 1.09	2.46K
[18] ⁴	SLH-DSA-256s	Artix-7 100T	51,130	74,576	27,436	22.5	250.00	- / -	19.30 / 0.14	529.51
[20]	SLH-DSA-256s	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	296.27M / 469.97K	2.96K / 4.70	29.79K
Pro.	SLH-DSA-256s	Artix-7 200T	15,213	27,958	5,687	4.5	220.00	53.13M / 93.70K	241.51 / 0.43	1.37K
[19]	SLH-DSA-256f	Artix-7 200T	10,416	7,672	5,422	8.0	150.00	7.41M / 337.49K	49.41 / 2.25	267.93
[18] ⁴	SLH-DSA-256f	Artix-7 100T	51,009	74,539	27,401	22.5	250.00	- / -	2.52 / 0.21	69.05
[20]	SLH-DSA-256f	Artix-7 100T	8,605	3,745	10,056	32.0	100.00	23.66M / 857.06K	236.60 / 8.57	2.38K
[8] ⁵	SLH-DSA-256f	Artix-7 100T	29,410	14,090	10,043	4.0	179.80	1.18M / 43.10K	6.58 / 0.24	66.12
Pro.	SLH-DSA-256f	Artix-7 200T	15,244	28,164	5,560	4.5	220.00	7.79M / 194.30K	35.42 / 0.88	196.95
[30] ⁶	SLH-DSA-128s	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	837.74M / 852.00K	5.58K / 5.68	38.80K
Pro.	SLH-DSA-128s	UltraScale+ XCZU9EG	14,347	24,047	3,057	2.5	150.00	55.08M / 51.29K	367.20 / 0.34	1.12K
[30] ⁶	SLH-DSA-128f	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	42.60M / 2.46M	283.99 / 16.39	1.97K
[8] ⁵	SLH-DSA-128f	UltraScale+ XCZU9EG	29,712	14,076	9,517	4.0	325.00	349.00K / 27.70K	1.07 / 0.09	10.78
Pro.	SLH-DSA-128f	UltraScale+ XCZU9EG	14,331	23,725	3,076	2.5	150.00	2.90M / 132.66K	19.34 / 0.88	59.50
[30] ⁶	SLH-DSA-192s	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	1538.60 / 1.30M	10.26K / 8.67	71.26K
Pro.	SLH-DSA-192s	UltraScale+ XCZU9EG	15,311	26,186	3,316	3.5	150.00	70.37M / 61.97K	469.15 / 0.41	1.56K
[30] ⁶	SLH-DSA-192f	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	72.13M / 3.73M	480.89 / 24.83	3.34K
[8] ⁵	SLH-DSA-192f	UltraScale+ XCZU9EG	29,712	14,076	9,517	4.0	325.00	571.90K / 40.50K	1.76 / 0.12	17.67
Pro.	SLH-DSA-192f	UltraScale+ XCZU9EG	15,842	26,186	3,316	3.5	150.00	4.23M / 186.24K	28.18 / 1.24	93.46
[30] ⁶	SLH-DSA-256s	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	1666.66 / 2.37M	11.11K / 15.79	77.19K
Pro.	SLH-DSA-256s	UltraScale+ XCZU9EG	15,311	28,247	3,318	4.5	150.00	53.13M / 93.70K	354.21 / 0.62	1.18K
[30] ⁶	SLH-DSA-256f	UltraScale+ XCZU9EG	22,488	10,601	6,947	-	150.00	179.50M / 4.80M	1.20K / 32.03	8.31K
[8] ⁵	SLH-DSA-256f	UltraScale+ XCZU9EG	29,712	14,076	9,517	4.0	325.00	1.18M / 43.10K	3.64 / 0.13	36.58
Pro.	SLH-DSA-256f	UltraScale+ XCZU9EG	15,273	28,050	3,422	4.5	150.00	7.79M / 194.30K	51.95 / 1.30	177.78

¹: Latency refers to the cycles of the whole Hypertree operation. ²: Delay=Latency/ F_{max} . Unit: ms.

³: Slice/CLB is calculated based on the formula $LUT/4 + FF/8 + DSP \times 102.4 + BRAM \times 232.4$ and $LUT/4 + FF/8 + DSP \times 51.2 + BRAM \times 116.2$ for 7-series and UltraScale+ board, respectively, according on [25] if the work didn't report it.

⁴: This work includes 1 DSP. ⁵: This work uses non-standardized parameter $w = 4$ instead of standard SLH-DSA parameter $w = 2^4 = 16$.

⁶: This work does not report its BRAM and DSP usage, we just do not include any BRAM or DSP in Slice/CLB calculation.

we demonstrate superior area efficiency (4,347 vs 9,268 slices) with $5.1\times$ better timing. On Artix-7, our ADP achieves 1.62K compared to 2.94K for [19] ($1.8\times$ improvement). Fast signature modes show reduced performance due to smaller h' values limiting triangle hash unit parallelization.

KECCAK-based. For SLH-DSA-128s, our design achieves 250.36 ms signature delay ($1.16\times$ improvement over [19]) and 0.23 ms verification ($2.2\times$ speed-up). Compared to [20], we achieve superior area efficiency (4,984 vs 10,056 slices) with $4.1\times$ better timing. Note that [18] supports only SHAKE256 and requires $4\text{-}5\times$ more resources, while [8] employs non-standard $w = 4$ vs standard $w = 16$. On UltraScale+, our design achieves 1.12K ADP vs 38.80K ($34.6\times$ improvement).

V. CONCLUSION

This paper proposed Trident, an innovative FPGA-based accelerator that addresses performance and resource challenges in

computationally intensive XMSS tree of SLH-DSA. The proposed architecture demonstrates significant advances through three key innovations: the triangle hash unit for parallel hash processing, an optimized memory caching scheme, and comprehensive evaluation for all NIST-specified parameter sets. The triangle hash unit brings the ability of concurrent execution of up to three hash operations, while the optimized KECCAK provides resource efficiency. For XMSS, the design achieves $25.2\times$ improvement in ADP compared to state-of-the-art. Extended evaluation of Hypertree (SLH-DSA-128s) achieves $34.6\times$ ADP improvement on UltraScale+ FPGA. Overall, the exploration of this work again demonstrates the high potentiality of FPGAs in PQC further deployment. Future work will focus on integrating the FORS signature component to achieve complete SLH-DSA acceleration, and exploring further optimizations for side-channel resistance in practical

cryptographic deployments. The presented XMSS/Hypertree accelerator provides a solid foundation for the FPGA-based PQC acceleration community to practically implement NIST-standardized post-quantum cryptographic schemes.

ACKNOWLEDGMENT

J. Xie was supported by Villanova College of Engineering Last-Year Ph.D. Fellowship and in part by NSF CNS-2453370.

REFERENCES

- [1] P. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, (Santa Fe, NM, USA), pp. 124–134, IEEE, 1994–11, 1994.
- [2] Information Technology Laboratory Computer Security Division, "Post-Quantum Cryptography." <https://csrc.nist.gov/projects/post-quantum-cryptography>, jan 2017.
- [3] Post-Quantum Cryptography (Selected Algorithms). <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms>.
- [4] P. Schwabe, "SPHINCS+." <https://sphincs.org/index.html>, aug 2024.
- [5] "National Institute of Standards and Technology. stateless Hash-Based Digital Signature Standard," Tech. Rep. Federal Information Processing Standard (FIPS) 205, U.S. Department of Commerce, 2024.
- [6] "National Institute of Standards and Technology. Secure Hash Standard (SHS)," Tech. Rep. Federal Information Processing Standard (FIPS) 180-4, U.S. Department of Commerce, 2015.
- [7] "National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," Tech. Rep. Federal Information Processing Standard (FIPS) 202, U.S. Department of Commerce, 2015.
- [8] T. Huang, J. Lu, D. Liu, Z. Luo, C. Cheng, A. Li, L. Chen, S. Yang, J. Zhang, and X. Li, "An Efficient and Reconfigurable Post-Quantum Crypto-Processor for SPHINCS+," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, pp. 2252–2262, May 2025.
- [9] S. Yoneda, K. Iwai, T. Matsubara, and T. Kurokawa, "FPGA Implementation of a Pipelined XMSS Leaf Accelerator," in *2024 Twelfth International Symposium on Computing and Networking Workshops (CANDARW)*, pp. 96–102, Nov. 2024.
- [10] R. Gonzalez, "Stateless Hash-Based Signatures for Post-Quantum Security Keys," 2025.
- [11] D. Kim, H. Choi, and S. C. Seo, "Parallel Implementation of SPHINCS+ With GPUs," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, pp. 2810–2823, June 2024.
- [12] J. Wu, Y. Yu, Z. Chen, H. Yang, C. Li, and Z. Liu, "CBSPX: A CUDA-based Batch Parallel Optimization of Post-Quantum Signature SPHINCS+," *IEEE Internet of Things Journal*, pp. 1–1, 2025.
- [13] W. Wang, B. Jungk, J. Wälde, S. Deng, N. Gupta, J. Szefer, and R. Niederhagen, "XMSS and Embedded Systems," in *Selected Areas in Cryptography – SAC 2019*, pp. 523–550, Springer, Cham, 2020.
- [14] J. Lopez-Valdivieso and R. Cumplido, "Design and Implementation of Hardware-Software Architecture Based on Hashes for SPHINCS+," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, pp. 54:1–54:22, Oct. 2024.
- [15] Z. Ye, X. Li, C. Wang, R. C. C. Cheung, and K. Huang, "RVSLH: Acceleration of Postquantum Standard SLH-DSA With Customized RISC-V Processor," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, pp. 1999–2003, July 2025.
- [16] S. Zhang, Y. Zhao, X. Yu, and J. Han, "Accelerating SPHINCS+ Using RISC-V Domain-Specific Processor on Multi-Core Systems," *Integrated Circuits and Systems*, pp. 1–9, 2025.
- [17] Q. Berthet, A. Upegui, L. Gantel, A. Duc, and G. Traverso, "An Area-Efficient SPHINCS+ Post-Quantum Signature Coprocessor," in *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 180–187, June 2021.
- [18] D. Amiet, L. Leuenberger, A. Curiger, and P. Zbinden, "FPGA-based SPHINCS+ Implementations: Mind the Glitch," in *2020 23rd Euromicro Conference on Digital System Design (DSD)*, pp. 229–237, Aug. 2020.
- [19] S. Deshpande, Y. Lee, C. Karakuzu, J. Szefer, and Y. Paek, "SPHINC-SLET: An Area-Efficient Accelerator for the Full SPHINCS+ Digital Signature Algorithm," *ACM Trans. Embed. Comput. Syst.*, Apr. 2025.
- [20] M.-J. O. Saarinen, "Accelerating SLH-DSA by Two Orders of Magnitude with a Single Hash Unit," in *Advances in Cryptology – CRYPTO 2024*, pp. 276–304, Springer, Cham, 2024.
- [21] J. Xie, W. Zhao, H. Lee, D. B. Roy, and X. Zhang, "Hardware circuits and systems design for post-quantum cryptography—A tutorial brief," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 3, pp. 1670–1676, 2024.
- [22] K. Gaj, "Implementation and Benchmarking of Round 2 Candidates in the NIST Post-Quantum Cryptography Standardization Process Using FPGAs." <https://csrc.nist.gov/CSRC/media/Projects/post-quantum-cryptography/documents/round-3/seminars/oct-2020-gaj-kris-presentation.pdf>, Oct. 2020.
- [23] P. Mohan, W. Wang, B. Jungk, R. Niederhagen, J. Szefer, and K. Mai, "ASIC Accelerator in 28 nm for the Post-Quantum Digital Signature Scheme XMSS," in *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pp. 656–662, Oct. 2020.
- [24] A. Wagner, F. Oberhansl, and M. Schink, "Extended version—to be, or not to be stateful: Post-quantum secure boot using hash-based signatures," *Journal of Cryptographic Engineering*, vol. 14, pp. 631–648, Nov. 2024.
- [25] AMD, "7 Series FPGAs Configurable Logic Block User Guide (UG474) (v1.9)." https://docs.amd.com/r/en-US/ug474_7Series_CLB/7-Series-FPGA-CLB-Resources, Apr. 2025.
- [26] J. Strömbergson, "Sha256." <https://github.com/secworks/sha256>, July 2025.
- [27] J. Strömbergson, "Sha512." <https://github.com/secworks/sha512>, June 2025.
- [28] Y. Cao, Y. Wu, W. Wang, X. Lu, S. Chen, J. Ye, and C.-H. Chang, "An Efficient Full Hardware Implementation of Extended Merkle Signature Scheme," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, pp. 682–693, Feb. 2022.
- [29] J. P. Thoma, D. Hartlief, and T. Güneysu, "Agile Acceleration of Stateful Hash-based Signatures in Hardware," *ACM Trans. Embed. Comput. Syst.*, vol. 23, pp. 29:1–29:29, Mar. 2024.
- [30] P. Karl, J. Schupp, and G. Sigl, "The Impact of Hash Primitives and Communication Overhead for Hardware-Accelerated SPHINCS+," in *Constructive Side-Channel Analysis and Secure Design*, pp. 221–239, Springer, Cham, 2024.